Team Reference Document

1. Environment & Debugging

Current computational power: 10^8 ops/sec. Generate all possible answers in background if large code size limit.

Python Errors:

- `RTE:` array size too small or recursion max depth.
- `EOFError, IndexError, SyntaxError`
- `KeyError:` access key not in dict.

Compare File:

```
python a.py < in.txt > out.txt && fc out.txt o_out.txt
```

Debug Wrapper:

```
def debug(f):
  def w(*a, **kw):
    print(f"In: {f.__name__} {a} {kw}")
    res = f(*a, **kw)
    print(f"Out: {f.__name__} = {res}")
    return res
  return w
```

2. Data Structures

Union-Find (Kruskal/Frogger)

```
class UF:
  def __init__(self, n):
    self.parents = [*range(n)]
    self.rank = [0]*n
  def find(self, x):
    if self.parents[x] != x:
      self.parents[x] = self.find(self.parents[x])
    return self.parents[x]
  def union(self, x, y):
    dx, dy = self.find(x), self.find(y)
    if dx == dy: return False
    if self.rank[dx] > self.rank[dy]:
      self.parents[dy] = dx
    elif self.rank[dx] < self.rank[dy]:
      self.parents[dx] = dy
    else:
      self.parents[dx] = dy
      self.rank[dy] += 1
    return True
```

Deque

```
from collections import deque
dq = deque([1, 2, 3])
dq.popleft(); dq.append(4)
```

Fenwick Tree / BIT

```
def getsum(bit, i):
  s = 0; i += 1
  while i > 0:
    s += bit[i]
    i -= i & (-i)
  return s
def updatebit(bit, n, i, v):
  i += 1
  while i <= n:
    bit[i] += v
    i += i & (-i)
def construct(arr):
  n = len(arr)
  bit = [0]*(n+1)
  for i in range(n):
    updatebit(bit, n, i, arr[i])
  return bit
```

3. Mathematics & Number Theory

Prime Sieve ($O(N \log \log N)$)

```
from math import sqrt
def prime(n):
  k = [True]*(n-2)
  for i in range(2, int(sqrt(n))+1):
    if k[i-2]:
      j = i*i
      while j < n:
        k[j-2] = False; j += i
  return [i+2 for i in range(n-2) if k[i]]
```

Fast Powering Algorithm ($O(\log P)$)

```
def fast(b, p):
  res = 1
  while p:
    if p&1: res *= b
    b *= b
    p >>= 1
  return res
```

Fibonacci $O(\log N)$ (Matrix Exponentiation Identity)

$F_{2k} = F_k(2F_{k+1} - F_k)$, $F_{2k+1} = F_{k+1}^2 + F_k^2$

```
memo = {0:0, 1:1, 2:1}
def fib(n, m=10**9+7):
  if n in memo: return memo[n]
  k = n//2
  if n%2 == 0:
    memo[n] = fib(k)*(2*fib(k+1)-fib(k))%m
  else:
    memo[n] = (fib(k+1)**2 + fib(k)**2)%m
  return memo[n]
```

Legendre's Formula

Number of times a prime P divides $N!$: $s = \sum_{k=1} \lfloor \frac{N}{P^k} \rfloor$

```
def legendre(num, factor):
  s = 0; curr = factor
  while num >= curr:
    s += num // curr
    curr *= factor
  return s
```

4. String Algorithms

KMP All Pattern Matching $O(N + M)$

```
def kmp(t, p):
  n, m = len(t), len(p)
  l = [0]*m; j = 0
  if m < 1: return [0]*(n>-1)
  if n < 1: return []
  for i in range(1, m):
    while j and p[i]!=p[j]: j=l[j-1]
    j += (p[i]==p[j]); l[i] = j
  i = j = 0; r = []
  while i < n:
    while i<n and j<m and t[i]==p[j]: i,j=i+1,j+1
    if j==m: r.append(i-j); j=l[j-1]
    elif j: j=l[j-1]
    else: i += 1
  return r
```

Trie

```
class Node:
  def __init__(self):
    self.child = dict()
    self.end = False
def build(strings):
  root = Node()
  for s in strings:
    cur = root
    for c in s:
      if c not in cur.child:
        cur.child[c] = Node()
      cur = cur.child[c]
    cur.end = True
  return root
```

Regex Guide (`import re`)

- **Syntax**: `*` 0+, `+` 1+, `?` 0/1, `{n,m}` $n \le x \le m$, lazy by `?` (`*?`, `+?`)
- **Classes**: `\d` digits, `\w` words, `\s` whitespace, `\b` word boundaries. Upper is negation.
- **Lookaround**: Positive `(?= )`, Negative `(?! )`. Lookbehind: Pos `(?<= )`, Neg `(?<! )`
- **Flags**: `re.I` ignorecase, `re.M` multiline, `re.X` verbose
- **Methods**: `re.findall(string, pat)`, `re.sub(pat, repl, str)`, `re.match` (start), `re.search` (first)

```
from re import finditer
[a.start() for a in finditer("k","kakakawww")]
```

5. Graph Theory

Dijkstra (Min-heap)

```
import heapq
def dijkstra(s, n, graph):
  # graph: dict/list of iterable (v, weight)
  wmap = [float('inf')]*n; wmap[s] = 0
  h = [(0, s)]
  while h:
    d, u = heapq.heappop(h)
    if d > wmap[u]: continue
    for v, w in graph[u]:
      if w+d < wmap[v]:
        wmap[v] = w+d
        heapq.heappush(h, (w+d, v))
  return wmap
```

Kahn's Topological Sort

```
from collections import deque
def kahn(n, graph, indeg):
  q = deque(c for c in indeg if indeg[c]==0)
  order = []
  while q:
    u = q.popleft()
    order.append(u)
    for v in graph[u]:
      indeg[v] -= 1
      if indeg[v] == 0:
        q.append(v)
  return order if len(order)==n else None
```

6. Geometry

Circle Intersection Context & Middle Area

Area enclosed by circles of radius a, b, c. Middle:

```
from math import sqrt, acos, pi
def calc_mid(a,b,c,a2,b2,c2):
  return a2*acos((c2-b2+a2)/(2*c*a)) + \
         b2*acos((c2-a2+b2)/(2*b*c)) - \
         sqrt((c-a+b)*(c+a-b)*(a+b-c)*(c+a+b))/2
```

Closest Pair ($O(N \log N)$ Sweep Equivalent)

```
from math import dist
def closest_pair(coords):
  coords.sort()
  d = dist(coords[0], coords[1])
  for i in range(2, len(coords)):
    p = i-1; new = coords[i]
    while p >= 0:
      pt = coords[p]
      if abs(new[0]-pt[0]) <= d:
        if abs(new[1]-pt[1]) <= 2*d:
          d = min(d, dist(new, pt))
      else: break
      p -= 1
  return d
```

7. Python Tips & Golfing

Bitwise Odd Check:

```
if a & 1: print("odd")
```

Tilde Golfing:

```
~x = -x - 1; -~x = x + 1; ~-x = x - 1
```

Reading all Input (Flat):

```
a,b,*c = map(int, open(0).read().split())
a = [*open(0)]
a = open(0).read().split()
```

List Flattening & Matrix Transpose:

```
import sum
A = sum([[1,2],[3,4]], []) # Flatten
T = [*zip(*A)]             # Transpose Matrix
```

Shortest Infinite Loops & Multi-tests:

```
for _ in [0]*int(input()): pass
for _ in ' '*int(input()): pass
```

Splat Operator (`*args`, `**kwargs`): Unpacks iterables: `add(*[1, 2, 3])`. Kwargs unpacks dicts: `fn(**{"name": "A"})`. Let arguments accept arbitrary items without strict parity.

LRUCache Memoization:

```
from functools import lru_cache
@lru_cache(maxsize=None)
def dp(i): pass
```